

Documentación Técnica

Sistema de Monitoreo de pH del Agua con ESP32 y Dashboard Web

Daniela Romero y Débora Vizama

July 10, 2026

Contents

1	Introducción	2
2	Componentes de Hardware Utilizados	2
2.1	Diagrama de conexión	2
3	Arquitectura del Software	3
4	Explicación de los Códigos, Método por Método	4
4.1	esp32/esp32_ph_sensor.ino	4
4.2	backend/serial_reader.py	5
4.3	backend/app.py	6
4.4	dashboard/streamlit_app.py	7
4.5	backend/simulate_data.py	8
5	Protocolo de Comunicación con el Sensor de pH	8
6	Estado Actual y Trabajo en Progreso	8
7	Conclusiones	8

1 Introducción

Este documento describe el diseño, los componentes de hardware y el funcionamiento del software del sistema de monitoreo de pH del agua desarrollado para el proyecto de feria científica. El objetivo del sistema es permitir que los estudiantes viertan una muestra de agua (destilada, con sal, con tierra, con limón, etc.) y observen en tiempo real, a través de un dashboard web, cómo cambian el pH y la temperatura, junto con una alerta visual tipo semáforo que indica si el agua se encuentra en un estado seguro, marginal o contaminado.

El sistema se compone de tres partes:

1. **Hardware:** microcontrolador ESP32 conectado a un sensor de pH industrial (protocolo RS-485) y a un sensor de temperatura sumergible (protocolo OneWire).
2. **Software de adquisición:** un programa embebido en el ESP32 que lee los sensores y los imprime por el puerto serial (USB), y un script en Python en la computadora que traduce esa información y la envía a un servidor.
3. **Software de visualización:** un servidor backend (Flask) que almacena las lecturas, y un dashboard web (Streamlit) que las muestra en gráficos dinámicos y alertas de color.

2 Componentes de Hardware Utilizados

2.1 Diagrama de conexión

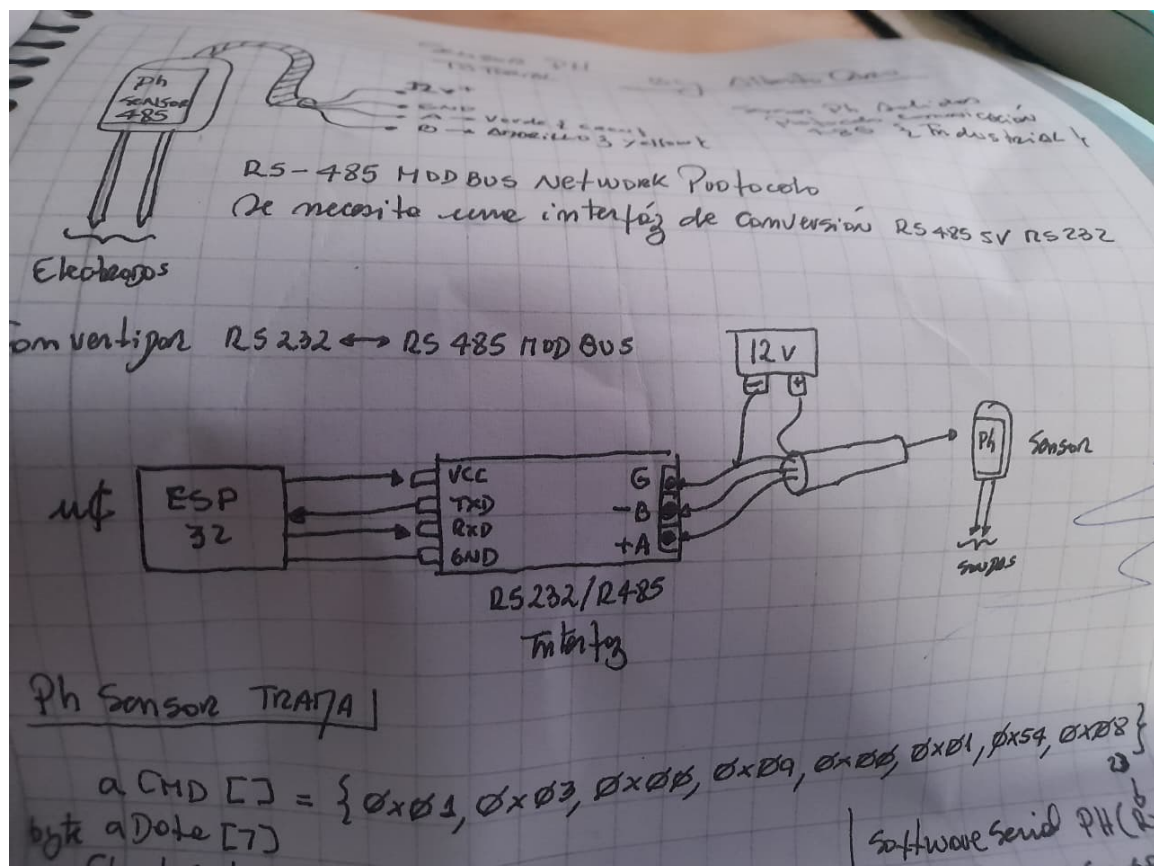


Figure 1: Diagrama de conexión del sensor de pH: ESP32, interfaz de conversión RS-232/RS-485, sensor de pH y fuente de 12V. Elaborado a partir de las notas originales del tutorial del sensor.

Componente	Descripción / Función
ESP32 (WROOM-32)	Microcontrolador principal. Se encarga de comunicarse con el sensor de pH por protocolo Modbus RS-485, leer el sensor de temperatura por OneWire, e imprimir los resultados por el puerto Serial (USB) hacia la computadora.
Sensor de pH industrial (RS-485)	Sensor sumergible con salida digital sobre RS-485 (protocolo Modbus). Entrega el valor de pH multiplicado por 10 en un registro que se lee mediante una trama de comando fija.
Interfaz conversora RS-232/RS-485	Módulo intermedio que adapta las señales del ESP32 (nivel TTL) al estándar eléctrico RS-485 que requiere el sensor de pH industrial.
Sensor de temperatura (DS18B20, OneWire)	Sensor sumergible de temperatura, conectado por el protocolo OneWire a un pin digital del ESP32 (pin 27 en este proyecto).
Transformador / fuente de alimentación 12V 2A	Fuente de poder externa que alimenta al sensor de pH y a la interfaz RS-485, ya que estos componentes requieren 12V, un voltaje mayor al que puede entregar el propio ESP32 (que trabaja a 3.3V/5V).
Cable USB	Utilizado para alimentar y programar el ESP32, y además como canal de comunicación serial (Serial Monitor) hacia la computadora, por donde se transmiten las lecturas de pH y temperatura que luego recoge el script <code>serial_reader.py</code> .
Computadora (laptop)	Ejecuta el backend (Flask), el script de lectura serial y el dashboard (Streamlit).

Table 1: Componentes de hardware del sistema

Nota: el sensor de pH y la interfaz RS-485 requieren alimentación de 12V, provista por el transformador. El ESP32 se alimenta de forma independiente por el cable USB conectado a la computadora, lo que además permite la transmisión de datos por puerto serial sin necesidad de una red WiFi.

3 Arquitectura del Software

El sistema sigue un flujo de datos lineal, sin necesidad de conexión a una red WiFi, ya que toda la comunicación entre el ESP32 y la computadora ocurre a través del mismo cable USB:

```

ESP32 (lee sensores)
  --> Serial/USB -->
serial_reader.py (lee el puerto y reenvía)
  --> HTTP POST (JSON) -->
app.py / Flask (guarda en SQLite, expone API)
  <-- HTTP GET (JSON) --
streamlit_app.py (dashboard: gráficos y semáforo)

```

Esta arquitectura desacopla la adquisición de datos (hardware) de la visualización (dashboard), lo que permite, por ejemplo, generar datos de prueba sin el sensor físico (ver `simulate_data.py`), o eventualmente reemplazar el mecanismo de transporte (por ejemplo, migrar a WiFi/MQTT) sin modificar el dashboard.

4 Explicación de los Códigos, Método por Método

4.1 esp32/esp32_ph_sensor.ino

Código embebido que corre directamente en el microcontrolador ESP32. Es responsable de leer los dos sensores físicos y reportar los valores por el puerto serial.

```
1 #include <OneWire.h>
2 #include <DallasTemperature.h>
3 #define nRx 23
4 #define nTx 22
5 const byte aCMD[] = {0x01,0x03,0x00,0x09,0x00,0x01,0x54,0x08};
6 const byte nGPTE = 27;
7 byte aData[7];
8 float fPH, fTE;
9 OneWire oneWire(nGPTE);
10 DallasTemperature st(&oneWire);
11 HardwareSerial PHS(2);
```

Listing 1: Declaraciones y variables globales

Explicación:

- `aCMD[]`: trama de comando Modbus fija que se envía al sensor de pH para solicitar la lectura de su registro interno. Sigue el protocolo Modbus RTU: dirección de esclavo, función, dirección de registro y CRC.
- `nGPTE = 27`: pin digital del ESP32 donde está conectado el sensor de temperatura OneWire.
- `PHS = HardwareSerial(2)`: se utiliza el segundo puerto serial de hardware del ESP32 (distinto del puerto USB de depuración) para comunicarse con la interfaz RS-485.
- `fPH, fTE`: variables donde se almacenan los últimos valores leídos de pH y temperatura.

```
1 void setup() {
2   Serial.begin(9600);
3   PHS.begin(9600, SERIAL_8N1, nRx, nTx);
4   st.begin();
5   delay(3000);
6 }
```

Listing 2: Método setup()

Explicación: inicializa el puerto Serial de depuración (el que verá la computadora por USB), inicializa el puerto Serial hacia la interfaz RS-485 con los parámetros de comunicación del sensor de pH (9600 baudios, 8 bits de datos, sin paridad, 1 bit de parada), e inicializa la librería del sensor de temperatura OneWire. El `delay(3000)` da tiempo a que los periféricos se estabilicen antes de comenzar a leer.

```
1 void loop() {
2   PHS.flush();
3   if (PHS.write(aCMD, sizeof(aCMD)) == 8) {
4     PHS.readBytes(aData, 7);
5   }
6   fPH = aData[4] / 10.0;
```

```
7 Serial.print("Soil PH: ");
8 Serial.println(fPH, 1);
9 ...
```

Listing 3: Método loop() - lectura de pH

Explicación: se envía la trama de comando `aCMD` al sensor de pH. Si la escritura fue exitosa (8 bytes enviados), se leen los 7 bytes de respuesta del sensor en el arreglo `aData`. El byte en la posición `aData[4]` contiene el valor de pH multiplicado por 10 (según la hoja de datos del sensor), por lo que se divide por 10.0 para obtener el valor real (por ejemplo, un valor crudo de 71 corresponde a pH 7.1). El resultado se imprime por el puerto Serial en un formato de texto fijo ("Soil PH: X.X"), que luego será interpretado por el script de Python.

```
1 st.requestTemperatures();
2 fTE = st.getTempCByIndex(0);
3 Serial.print("Temperatura: ");
4 Serial.print(fTE);
5 Serial.println(" grados C");
6 delay(10000);
7 }
```

Listing 4: Método loop() - lectura de temperatura

Explicación: se solicita al sensor OneWire una nueva conversión de temperatura (`requestTemperatures()`) y se lee el resultado del primer sensor detectado en el bus (`getTempCByIndex(0)`). El valor se imprime de la misma forma que el pH, en texto plano por el puerto Serial. El `delay(10000)` espacia las lecturas cada 10 segundos.

4.2 backend/serial_reader.py

Script en Python que corre en la computadora. Actúa como puente entre el puerto USB (donde el ESP32 imprime texto) y el servidor backend (que espera datos en formato JSON vía HTTP).

```
1 def listar_puertos():
2     puertos = serial.tools.list_ports.comports()
3     for p in puertos:
4         print(f" {p.device} - {p.description}")
```

Listing 5: listar_puertos()

Explicación: utiliza la librería `pyserial` para listar todos los puertos seriales disponibles en el sistema operativo (por ejemplo COM3 en Windows), junto con una descripción del dispositivo. Permite al usuario identificar a qué puerto está conectado el ESP32 antes de iniciar la lectura.

```
1 def leer_y_enviar(puerto: str):
2     with serial.Serial(puerto, BAUDRATE, timeout=2) as ser:
3         while True:
4             linea = ser.readline().decode("utf-8", errors="ignore").strip()
5             match_ph = re.search(r"Soil PH:\s*([\d.]+)", linea)
6             match_temp = re.search(r"Temperatura:\s*([\d.]+)", linea)
7             if match_ph:
8                 ph_actual = float(match_ph.group(1))
9             if match_temp:
10                 temp_actual = float(match_temp.group(1))
11             if ph_actual is not None and temp_actual is not None:
12                 enviar_al_backend(ph_actual, temp_actual)
```

Listing 6: leer_y_enviar(puerto) - núcleo del script

Explicación: abre una conexión al puerto serial indicado con la misma velocidad (baudrate) configurada en el ESP32 (9600). En un bucle infinito, lee línea por línea el texto que imprime el ESP32. Utiliza expresiones regulares (`re.search`) para extraer el número que sigue a "Soil

PH:" o a "Temperatura:". Como el ESP32 imprime ambos valores en líneas separadas dentro del mismo ciclo de `loop()`, el script acumula ambos valores y, recién cuando tiene el par completo, lo envía al backend, reiniciando luego las variables para esperar el siguiente ciclo.

```

1 def enviar_al_backend(ph: float, temp: float):
2     r = requests.post(
3         BACKEND_URL,
4         json={"ph": ph, "temperatura": temp},
5         timeout=3,
6     )

```

Listing 7: `enviar_al_backend(ph, temp)`

Explicación: realiza una petición HTTP POST al backend Flask, enviando los datos codificados en formato JSON. Este es el punto de integración entre el mundo del hardware (serial) y el mundo del dashboard web (HTTP).

4.3 backend/app.py

Servidor backend construido con el microframework Flask. Su responsabilidad es recibir, almacenar y exponer las lecturas de los sensores.

```

1 def init_db():
2     conn = sqlite3.connect(DB_PATH)
3     conn.execute("""
4         CREATE TABLE IF NOT EXISTS lecturas (
5             id INTEGER PRIMARY KEY AUTOINCREMENT,
6             ph REAL NOT NULL,
7             temperatura REAL,
8             timestamp TEXT NOT NULL
9         )
10    """)
11    conn.commit()
12    conn.close()

```

Listing 8: `init_db()`

Explicación: crea (si no existe) una base de datos SQLite local con una tabla `lecturas`, que almacena cada lectura con un identificador autoincremental, el valor de pH, la temperatura y la marca de tiempo en que fue recibida. SQLite se eligió por ser un motor de base de datos embebido, que no requiere instalar ni configurar un servidor de base de datos externo.

```

1 @app.route("/api/lectura", methods=["POST"])
2 def recibir_lectura():
3     data = request.get_json(force=True, silent=True)
4     if not data or "ph" not in data:
5         return jsonify({"error": "falta el campo 'ph'"}), 400
6     ph = float(data["ph"])
7     temperatura = float(data.get("temperatura", 0))
8     timestamp = datetime.now().isoformat()
9     conn.execute("INSERT INTO lecturas (ph, temperatura, timestamp) VALUES (?, ?, ?)", ...)

```

Listing 9: `recibir_lectura()` - endpoint POST `/api/lectura`

Explicación: define el punto de entrada HTTP que recibe los datos enviados por `serial_reader.py` (o por `simulate_data.py`). Valida que el campo `ph` esté presente; de lo contrario responde con un error HTTP 400. Si los datos son válidos, genera una marca de tiempo con el instante actual del servidor y guarda el registro completo en la base de datos.

```

1 @app.route("/api/lecturas", methods=["GET"])
2 def obtener_lecturas():
3     limite = int(request.args.get("limite", 100))

```

```

4     filas = conn.execute(
5         "SELECT ph, temperatura, timestamp FROM lecturas ORDER BY id DESC LIMIT
        ?",
6         (limite,),
7     ).fetchall()
8     lecturas = [dict(f) for f in reversed(filas)]
9     return jsonify(lecturas)

```

Listing 10: obtener_lecturas() - endpoint GET /api/lecturas

Explicación: este es el endpoint que consulta el dashboard. Devuelve las últimas N lecturas almacenadas (100 por defecto), ordenadas cronológicamente de la más antigua a la más reciente (por eso se invierte el orden con `reversed`, ya que la consulta SQL las trae de la más nueva a la más antigua para poder aplicar el LIMIT correctamente).

4.4 dashboard/streamlit_app.py

Aplicación web construida con Streamlit, responsable de la visualización final que ven los usuarios.

```

1 def clasificar_ph(ph: float):
2     if PH_MIN_SEGURO <= ph <= PH_MAX_SEGURO:
3         return "SEGURA", "#2ecc71"
4     elif PH_MIN_MARGINAL <= ph <= PH_MAX_MARGINAL:
5         return "MARGINAL", "#f1c40f"
6     else:
7         return "CONTAMINADA", "#e74c3c"

```

Listing 11: clasificar_ph(ph) - lógica del semáforo

Explicación: implementa la regla de negocio central del proyecto: clasifica el estado del agua según el valor de pH, devolviendo una etiqueta de texto y un color hexadecimal asociado (verde, amarillo o rojo). Los umbrales (`PH_MIN_SEGURO = 6.5`, `PH_MAX_SEGURO = 8.5`, etc.) están definidos como constantes al inicio del archivo y pueden ajustarse fácilmente sin modificar el resto de la lógica.

```

1 def obtener_lecturas(limite=100):
2     r = requests.get(f"{BACKEND_URL}/api/lecturas", params={"limite": limite},
3         timeout=3)
4     return pd.DataFrame(r.json())

```

Listing 12: obtener_lecturas(limite) - consumo de la API

Explicación: realiza una petición HTTP GET al backend Flask y convierte la respuesta JSON en un `DataFrame` de `pandas`, estructura tabular que facilita graficar series de tiempo directamente con Streamlit.

```

1 while True:
2     df = obtener_lecturas()
3     with placeholder.container():
4         ultima = df.iloc[-1]
5         estado, color = clasificar_ph(ultima["ph"])
6         st.metric("pH actual", f"{ultima['ph']:.2f}")
7         st.line_chart(df.set_index("timestamp")["ph"])
8         time.sleep(REFRESCO_SEGUNDOS)

```

Listing 13: Bucle principal del dashboard

Explicación: es el ciclo de actualización del dashboard. Cada `REFRESCO_SEGUNDOS` (2 segundos por defecto), vuelve a consultar el backend, obtiene el último valor registrado, calcula su clasificación de color, y redibuja tanto los indicadores numéricos como los gráficos de línea histórica, usando el contenedor `placeholder` de Streamlit para actualizar la pantalla sin necesidad de recargar la página completa.

4.5 backend/simulate_data.py

Script auxiliar, no dependiente del hardware, que genera datos de pH y temperatura aleatorios y los envía al backend con la misma estructura que `serial_reader.py`. Se utiliza para probar y demostrar el funcionamiento del dashboard cuando no se dispone del sensor físico conectado.

5 Protocolo de Comunicación con el Sensor de pH

El sensor de pH utilizado se comunica bajo el protocolo **Modbus RTU sobre RS-485**, un estándar ampliamente usado en sensórica industrial. La trama de solicitud enviada es:

0x01	0x03	0x00	0x09	0x00	0x01	0x54	0x08
Dirección	Función	Registro inicial	Cantidad		CRC		

El sensor responde con una trama de 7 bytes, de la cual el byte en la posición 4 contiene el valor de pH multiplicado por 10. Esta comunicación de bajo nivel requiere una interfaz conversora RS-232/RS-485, ya que el ESP32 trabaja con señales lógicas TTL y no puede comunicarse directamente con el bus RS-485 diferencial que utiliza el sensor.

6 Estado Actual y Trabajo en Progreso

El sistema actualmente cumple con el objetivo mínimo viable: lectura del sensor de pH, transmisión al backend y visualización en un dashboard web accesible desde un navegador de escritorio. Sin embargo, el proyecto se encuentra en desarrollo activo, y se identificaron las siguientes mejoras futuras:

- **Dashboard responsivo / versión móvil:** actualmente el dashboard está optimizado para ser visto en un navegador de escritorio. Se planea adaptar el diseño para que también pueda visualizarse correctamente desde un teléfono celular (diseño responsivo), permitiendo monitorear el estado del agua desde cualquier dispositivo conectado a la misma red, sin depender de estar frente a la computadora que ejecuta el servidor.
- **Conectividad WiFi opcional:** evaluar una versión alternativa del firmware del ESP32 que envíe los datos por WiFi directamente (sin depender del cable USB), lo que permitiría ubicar el sensor de forma remota, lejos de la computadora.
- **Persistencia y exportación de datos:** agregar la posibilidad de exportar el historial de lecturas (por ejemplo, a formato CSV) para análisis posterior en herramientas de hoja de cálculo.
- **Alertas activas:** además del semáforo visual en el dashboard, evaluar la incorporación de un LED físico (rojo/verde) conectado directamente al ESP32, que refleje el estado del agua en el mismo punto de medición, sin depender de mirar la pantalla.
- **Incorporación de sensores adicionales:** integrar en el mismo dashboard los sensores de turbidez y de conductividad eléctrica / TDS contemplados originalmente en el diseño del proyecto, para tener una caracterización más completa del agua.

7 Conclusiones

El sistema desarrollado logra conectar de manera efectiva un sensor de pH industrial, normalmente utilizado en contextos agrícolas o industriales, con una interfaz visual moderna, interactiva

y comprensible para un público no especializado. La arquitectura modular (adquisición vía serial, backend independiente, dashboard web) permite que cada componente evolucione por separado —por ejemplo, agregar soporte móvil o nuevos sensores— sin necesidad de rediseñar el sistema completo.